

SENSORMETER-FAMILIE

Systemübersicht — Funktionsumfang, Zusammenspiel, Zabbix-Anbindung und Betrieb der drei Firmware-Projekte

v0.9.0-rc2 · Beta

3 ESP32-Firmwares, ein Ursprung

SNMP · Zabbix · lokales OTA

github.com/peterhagelhof7-cmd

- 1. Die drei Projekte im Überblick
- 2. Zusammenspiel & Architektur
- 3. Zabbix-Integration
- 4. Flashen & Updates
- 5. Gemeinsame Design-Entscheidungen
- 6. Repositories & Dokumentation

1 Die drei Projekte im Überblick

Sensormeter ist der Ursprung der Familie: ein Ethernet-first-Messgerät auf WT32-ETH01 mit bis zu zwei DHT-Sensoren. **Sensormeter WLAN** ist die kabellose, günstigere Abwandlung auf einem generischen ESP32-DevKit mit genau einem Sensor — gleiche Architektur, reduzierter Funktionsumfang. **Sensormeter Display** steht als Drittes daneben: ein Touchscreen-Gerät, das keine eigenen Messstationen ersetzt, sondern die Werte der beiden Geschwister per SNMP zusammenführt, zusätzlich einen eigenen internen Sensor sowie Netzwerk-Ping-Ziele überwacht und alles auf einem Blick darstellt.

Projekt	Board	Netzwerk	Sensoren	Rolle & Schnittstellen
Sensormeter	WT32-ETH01	LAN (+ optional WLAN), Fallback-WLAN „installer“	1–2× DHT (intern/extern), ganzzahlige Kalibrierkorrektur	SNMP-Agent (read-only), Webserver (öffentliches Dashboard + Admin-Bereich), Syslog
Sensormeter WLAN	ESP32-WROOM-32 DevKit	WLAN-only, Fallback-WLAN „installer“	1× DHT22, ganzzahlige Kalibrierkorrektur	SNMP-Agent (read-only), Webserver, OLED-Statusanzeige, Syslog
Sensormeter Display	HW-458B (ESP32 + 2,8" Touch-TFT)	WLAN	1× DHT11 (intern) + SNMP-Abfrage der Geschwister + Ping-Ziele	SNMP-/Ping-Client (kein eigener Agent), Touch-UI, eigener Webserver (öffentliches Dashboard + Admin), konfigurierbare Warnschwellwerte

GEMEINSAMES PRINZIP

Messwerte sind immer read-only per SNMP abrufbar, jede Konfiguration läuft ausschließlich über das eigene, lokale Webinterface des jeweiligen Geräts — keine Cloud-Anbindung, kein externer Dienst ist für den Betrieb notwendig.

2 Zusammenspiel & Architektur

Sensormeter und Sensormeter WLAN arbeiten unabhängig voneinander: jedes Gerät misst an seinem Standort und beantwortet SNMP-v1/v2c-GET-Anfragen unter derselben Basis-OID `.1.3.6.1.4.1.99999.x` — gleiche Struktur-Systematik, damit beide mit demselben Werkzeugsatz (auch Zabbix, siehe Abschnitt 3) ausgelesen werden können. Sensormeter Display tritt dazu als reiner Betrachter auf: es fragt bis zu mehrere konfigurierte Sensormeter-Ziele per SNMP ab, ergänzt sie um seinen eigenen internen DHT11-Sensor sowie beliebige Ping-Ziele, und wertet für alle zusammen konfigurierbare Warnschwellwerte aus (Temperatur/Feuchte in vollen °C/%, Ping-Latenz in ms) — bei Überschreitung färbt sich der gesamte Bildschirm rot, mit Angabe, welche Quelle den Alarm ausgelöst hat.



2.1 Was die drei Firmwares technisch verbindet

Konvention	Ausprägung
Architektur	Manager-Klassen mit <code>begin()</code> / <code>loop()</code> , mutex-geschützte zentrale Datenhaltung (<code>DataManager</code> / <code>SettingsManager</code>) für den parallelen Zugriff aus Sensor-Loop und asynchronem Webserver-Task
SNMP-Basis-OID	<code>.1.3.6.1.4.1.99999.x</code> , SNMP v1/v2c, read-only, Community konfigurierbar
Fallback-Netz	Nach 5 Minuten ohne Netzwerkverbindung: automatischer Wechsel auf Recovery-WLAN <code>installer</code> / <code>installer</code>
Konfigurations-Persistenz	Sensormeter/WLAN: <code>config.xml</code> auf LittleFS (tinyxml2); Display: NVS (Preferences) — jeweils mit sicherem Schreib-Umschreib-Mechanismus
Kalibrierkorrektur	Ganzzahliger °C-/%-Versatz je Sensor, an einer einzigen Stelle im Messpfad angewendet — wirkt dadurch automatisch gleichermaßen auf Anzeige, SNMP-Werte und Verlauf; letzter Kalibrierzeitpunkt wird persistent gespeichert
Verlaufsspeicherung	Sensormeter/WLAN: 7-Tage-Stundenwerte in <code>/history.csv</code> ; Display: 12-Stunden-Verlauf ebenso in <code>/history.csv</code> — beide persistent auf LittleFS, überstehen also einen Neustart
mDNS	Alle drei erreichbar unter <code><geraetenname>.local</code> im lokalen Netz, ohne die IP-Adresse nachschlagen zu müssen
Web-Login & Design	Benutzer <code>admin</code> , projektspezifisches Passwort; einheitliche Weboberfläche (Navy/Orange/Cream) über alle drei Projekte hinweg, jeweils mit öffentlichem, nicht passwortgeschütztem Live-Dashboard vor dem Admin-Bereich

3 Zabbix-Integration

Weder das Lastenheft noch das Pflichtenheft fordern Zabbix — reine SNMP-Erreichbarkeit hätte genügt. Da ein früherer Prototyp aber bereits ein funktionierendes Zabbix-Template mitbrachte, wurde die Anbindung bewusst als zusätzliches, optionales Monitoring-Feature weitergeführt und an die finale OID-Struktur angepasst. Im Ergebnis ergibt sich ein durchgängiges Überwachungsbild über die ganze Familie, ganz ohne zusätzliche Software auf den Geräten oder einem Gateway:

Projekt	Zabbix-Methode	Template	Wichtige Trigger
Sensormeter	SNMP v1/v2c, Port 161, read-only	<code>zabbix-template-sensormeter.yaml</code>	Temperatur/Feuchte zu hoch (Sensor 1/2), freier Heap niedrig, keine Daten seit 10 Minuten
Sensormeter WLAN	SNMP v1/v2c, dieselbe OID-Familie (ein Sensor statt zwei)	als generischer SNMP-Host anlegbar — dediziertes YAML-	analog zu Sensormeter, manuell konfigurierbar

		Template steht noch aus	
Sensormeter Display	kein SNMP-Agent (reiner Client) — agentenlose ICMP-Erreichbarkeitsprüfung	<code>zabbix-template-sensormeter-display.yaml</code>	Display nicht erreichbar, hoher Paketverlust, hohe Antwortzeit

WARUM DAS DISPLAY ANDERS ÜBERWACHT WIRD

Das Display selbst stellt keine Messwerte über das Netzwerk bereit — es ist Konsument, nicht Quelle. Zabbix würde dort nichts finden, was es per SNMP abfragen könnte. Die eigentlich interessanten Werte (DHT-Messungen der Sensormeter-Geräte) überwacht Zabbix bereits direkt an der Quelle; am Display selbst lohnt sich nur die Frage „lebt es noch, ist es erreichbar“ — genau das deckt die ICMP-Simple-Check-Vorlage ab, ganz ohne Agent auf dem Gerät.

SNMP ist bei allen Geräten strikt read-only: Zabbix kann Werte auslesen, aber am Gerät nichts verändern. Der Community-String ist pro Gerät über die jeweilige Einstellungsseite konfigurierbar (Default `public`); weicht er vom Zabbix-Default ab, genügt es, das Host-Makro `{SNMP_COMMUNITY}` anzupassen. Mehrere Geräte desselben Typs bekommen jeweils einen eigenen Zabbix-Host mit demselben Template, unterschieden nur durch IP-Adresse und Systemnamen.

4 Flashen & Updates

Zwei bewusste Entscheidungen machen Ersteinrichtung und spätere Updates über die ganze Familie hinweg ungewöhnlich einfach, gerade weil sie den Aufwand aus dem Wartungsalltag heraus in eine einmalige Vorbereitung verschieben:

4.1 Ersteinrichtung: ein Skript für alle drei Projekte

`scripts/flash.ps1` liegt identisch in allen drei Repositories. Es fragt (oder erhält per Parameter), welches der drei Projekte geflasht werden soll, und übernimmt danach automatisch:

- Installation von Python, Git und PlatformIO, falls nicht vorhanden — per `winget` / `pip`
- funktionale statt reine PATH-Prüfung (erkennt z. B. Windows' leeren „python“-Store-Alias als das, was er ist: keine echte Installation)
- Klonen des gewählten Repos (oder `git pull` bei vorhandenem, sauberem Checkout) — fehlende Geschwister-Repos werden bei Bedarf automatisch daneben mitgeklont
- Anlegen von `firmware/include/config.h` aus der Vorlage, falls nötig (wird nie überschrieben)
- Bauen (`pio run`) und Flashen (`pio run --target upload`) über den per USB angeschlossenen COM-Port

```
.\flash.ps1 -Project display
.\flash.ps1 -Project sensormeter
.\flash.ps1 -Project wlan
```

Parameter	Wirkung
<code>-Project</code>	<code>sensormeter</code> , <code>wlan</code> oder <code>display</code> — ohne Angabe fragt das Skript interaktiv
<code>-Port</code>	seriellen Port explizit angeben (z. B. <code>COM5</code>), falls die automatische Erkennung fehlschlägt
<code>-RepoPath</code>	Zielordner für einen neuen Checkout, falls abweichend vom Standard
<code>-SkipUpload</code>	nur bauen, nicht flashen — zur Kontrolle ohne angeschlossenes Board

4.2 Laufende Updates: lokales OTA, ganz ohne PC

Nach der Ersteinrichtung genügt für neue Firmware-Versionen ein Browser: Alle drei Weboberflächen bieten im Admin-Bereich einen Upload-Button, der eine `.bin`-Datei direkt auf das Gerät überträgt und automatisch neu startet — kein erneuter USB-Anschluss, kein PlatformIO nötig.

DIE ENTSCHEIDUNGEN DAHINTER

Zwei Design-Entscheidungen aus der Praxis machen diesen Weg so unkompliziert:

- **Standard-Partitionstabelle unverändert übernommen:** Die Standard-ESP32-Partitionierung bringt bereits zwei OTA-fähige App-Partitionen (`ota_0` / `ota_1`) mit — kein Anpassen des Flash-Layouts nötig, OTA funktionierte von Anfang an.
- **Cloud-Update-Weg bewusst wieder entfernt:** Ein automatischer GitHub-Versioncheck mit Direktinstall war zunächst umgesetzt, kostete aber allein durch den nötigen HTTPS-Client rund 168 KB Flash — zu viel Risiko für die späteren Ausbaustufen (SNMP, Syslog). Übrig blieb der ohnehin vorgesehene lokale `.bin` -Upload: schlanker Code, funktioniert auch ganz ohne Internetzugang.

5 Gemeinsame Design-Entscheidungen

Auswahl der Entscheidungen mit dem größten Effekt auf Wartbarkeit und Betrieb, jeweils identisch oder sinngemäß gleich in allen drei Projekten umgesetzt (Details siehe `docs/entscheidungen.md` im jeweiligen Repo):

- **Öffentliches Live-Dashboard vor dem Admin-Bereich:** aktuelle Werte ohne Login einsehbar, Konfiguration bleibt passwortgeschützt — bewusste Trennung von „nachsehen“ und „verändern“.
- **Kein Wildcard-Update auf Kosten der Robustheit:** Konfigurations- Schreibvorgänge laufen über eine temporäre Datei mit anschließendem Umbenennen, damit ein Stromausfall mitten im Speichern nicht die bestehende, funktionierende Konfiguration zerstört.
- **Kalibrierung mit Nachweis:** Korrekturwerte wirken an einer einzigen Stelle im Messpfad (nicht mehrfach verstreut) und der Zeitpunkt der letzten tatsächlichen Änderung wird persistent mitgeführt — sichtbar auf der jeweiligen Einstellungsseite.
- **Verlauf übersteht einen Neustart:** sowohl der 7-Tage- als auch der 12-Stunden-Verlauf werden nicht mehr nur im RAM gehalten, sondern bei jeder Aktualisierung auf LittleFS persistiert.
- **Erreichbarkeit ohne IP-Adresse:** mDNS macht alle drei Geräte unter einem sprechenden Namen (`<name>.local`) statt einer zu merkenden IP erreichbar.

6 Repositories & Dokumentation

Projekt	Repository	Weiterführende Dokumentation
Sensormeter	github.com/peterhagelhof7-cmd/sensormeter	README, <code>docs/sensormeter-onepager.pdf</code> , <code>docs/ZABBIX.md</code> , <code>docs/entscheidungen.md</code>
Sensormeter WLAN	github.com/peterhagelhof7-cmd/sensormeter-wlan	README, <code>docs/admin-guide.pdf</code> , <code>docs/sensormeter-wlan-onepager.pdf</code> , <code>docs/entscheidungen.md</code>
Sensormeter Display	github.com/peterhagelhof7-cmd/sensormeter-display	README, <code>docs/admin-guide.pdf</code> , <code>docs/sensormeter-display-onepager.pdf</code> , <code>docs/ZABBIX.md</code> , <code>docs/entscheidungen.md</code>

Jedes Repository enthält außerdem `docs/projektfamilie.html` mit einer kompakten Architekturgrafik der ganzen Familie sowie ein eigenes `docs/entscheidungen.md` als lückenloses Entscheidungsprotokoll für alle Auslegungsfragen, die Lastenheft/ Pflichtenheft offen ließen.